Practical Training

AT PROGINEER

Ahmad Saddouq

First Week

This week, I started by learning about Scala programming language, and Rand told me to practice this language very well, so I started by watching a course that I chose from YouTube, and I suggested it to Rand and she told me it was good.

Name of the course: **Scala tutorial – Scala at Light Speed.**

Link of the course on YouTube: **https://www.youtube.com/watch?v=-8V6bMjThNo&list=PLmtsMNDRUoBxryRX4wiwrTZ661xcp6VPM**

In this course, I learned a lot about Scala, and this programming language combines object-oriented and functional programming in one concise. And finally, after practicing it, I solved many "problem-solving" using this language on LeetCode.

SECOND WEEK

I started to learn about the apache spark server, and also, the trainer recommended that I should download the Apache course which was the first step to learning more about apache spark the course name was Apache Spark with Scala Hands On with Big Data, and also he sent a book to me to read from it, the book was about apache-spark to, but, its main goal to achieve the best performance while dealing with apache-spark.

The name of the book is **High-Performance Spark_ Best Practices for Scaling and Optimizing Apache Spark.**

This book covers a lot of important things such as:

-Introduction to High-Performance Spark.

-How Spark works.

-Data Frames, Datasets, and SparkSql.

-Joins (Sql and Cores).

-Effective Transformations.

-Working with key/value data.

-Going beyond Scala.

-Testing and Validation.

THIRD WEEK

This week, I was able to deal with Scala and Spark together and now I can deal with them without any problems, so I practiced many exercises, these exercises were great to get ready to get into solving bigger problems and tasks. **And for the tasks, I will send them to you as a file**. But here for the tasks, I learned how to use the RDD and also I learned how to use the SparkSql queries to make the appropriate operations on the file (CSV/JSON) files, such as using the average and summation operations and also using the filtering operation and applying them to the used file.

FOURTH WEEK

This week I started to learn about the Cassandra database which Rand recommended to me. And for the version, this is the version I used.

```
C:\Program Files (x86)\sbt\bin
C:\apache-cassandra-3.11.4\bin
C:\Java\jdk1.8.0_301\bin
C:\Python27
C:\spark-3.3.2-bin-hadoop3\bin
C:\Program Files\Java\jdk1.8.0_361\bin
```

I used **Apache-Cassandra-3.11.4 and also I installed JDK1.8.0_301** which is compatible with the version of the installed Database. Also, I installed **Hadoop3 and sbt**.

And for the Spark version I used which it's also important to be compatible with the version of the Cassandra Database was Spark 3.3.2 as shown below.

```
C:\Users\Pavilion>spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel
23/05/25 13:21:07 WARN NativeCodeLoader: Unable to load native-hadoop library for your pla
asses where applicable
Spark context Web UI available at http://DESKTOP-EBRDOKS:4040
Spark context available as 'sc' (master = local[*], app id = local-1685010069419).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.3.2
      /_/

Using Scala version 2.12.15 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_361)
Type in expressions to have them evaluated.
Type :help for more information.

scala> 23/05/25 13:21:19 WARN ProcfsMetricsGetter: Exception when trying to compute pagesi
rocessTree metrics is stopped
```

And also I linked the Database with the Apache Spark server which I found it hard at the begging but I solved all the problems that I faced and I connected them all together and these are the libraries that I used to complete the connection process.

```
ThisBuild / version := "0.1.0-SNAPSHOT"

ThisBuild / scalaVersion := "2.12.15"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-mllib" % "3.3.2",
  "org.apache.spark" %% "spark-core" % "3.3.2",
  "org.apache.spark" %% "spark-sql" % "3.3.2",
  "com.datastax.spark" %% "spark-cassandra-connector" % "3.3.0",
  "com.datastax.oss" % "java-driver-core" % "4.15.0",
  "com.datastax.oss" % "native-protocol" % "1.5.1"



)
```

FIFTH WEEK

This week I started a new course, this course talks deeply about Apack Spark using Scala, and I recommended this course to Rand she told me to start with it.

Course name: Apacke Spark with Scala Hands On with Big Data.

This course has 51 videos, and these videos talks about Spark and how to parse files, and also it's more detailed and it gives hard tasks, such as using **Joins** between files. And how to parse CSV files and how to apply the operations on them.

And also I learned more about RDD (Resilient Distributed Data), Spark Sql, Data Frames and Advanced examples of Spark Programs.

And finally I learned how to run Spark on a Cluster, which it was hard for me to understand the internal process, but finally I got the time to read more about this topic, and I found It interesting and It helped me to understand the internal process more and more

SIXTH WEEK

This week, I started to learn how to parse JSON files, as shown below.

This example computes the Total Revenue for Each Customer, First, I initialized the Spark

```scala
object Revenue_Jsonf {
  def main(args: Array[String]) {
    val spark = SparkSession.builder.master( master = "local[*]").appName( name = "Revenue_Jsonf").getOrCr
    //    val ssc = new StreamingContext(spark.sparkContext, Seconds(10))
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._
    import spark.sql

    val Reading_Configuration = spark.read.option("multiline", "true").json( path = "C:\\TestFiles\\JSON

    // Show the DataFrame
    val Revenue = Reading_Configuration.
      withColumn( colName = "revenue", col( colName = "quantity") * col( colName = "price"))

    val Customer_Revenue = Revenue.
      groupBy( col1 = "customer_id").
      agg(sum( columnName = "revenue").
        alias( alias = "Total_Revenue_For_Each_Customer"))

    Customer_Revenue.show()
```

Session variable and the Spark Session **is the entry point to Spark SQL. It is one of the very first objects you create while developing a Spark SQL application.**

Then after that, I initialized the Spark Context variable, and it's important to know that the program **must** have only one Spark Context variable to use for the program.

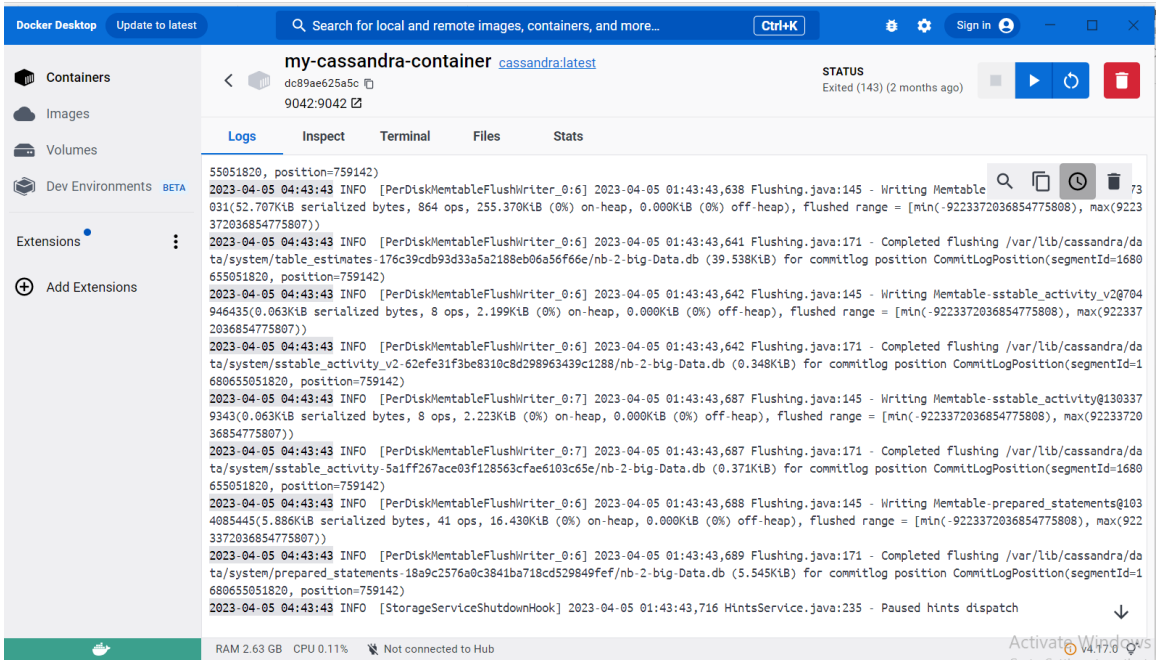
Then for the **Reading_Configuration** variable, here I used it to read the file which is a "JSON file".

Then I applied the Revenue variable, and here I applied it to calculate the **Revenue by multiplying the Quantity by Price,** and here I used the **Reading_Configuration variable to apply the operation which helped me to access the columns of the file.**

And finally, I applied some operations, such as group By, agg (sum ()), alias, and finally "show ()" operation to load the Data and display it on the console and show the final result.

SEVENTH WEEK

This week, I started to use Cassandra Data Base, and for this, I installed Docker and I installed the Data Base on it.



In this figure, we can see that we can run the database on Docker just by clicking on the run button, then here we can go to the terminal to apply some commands.

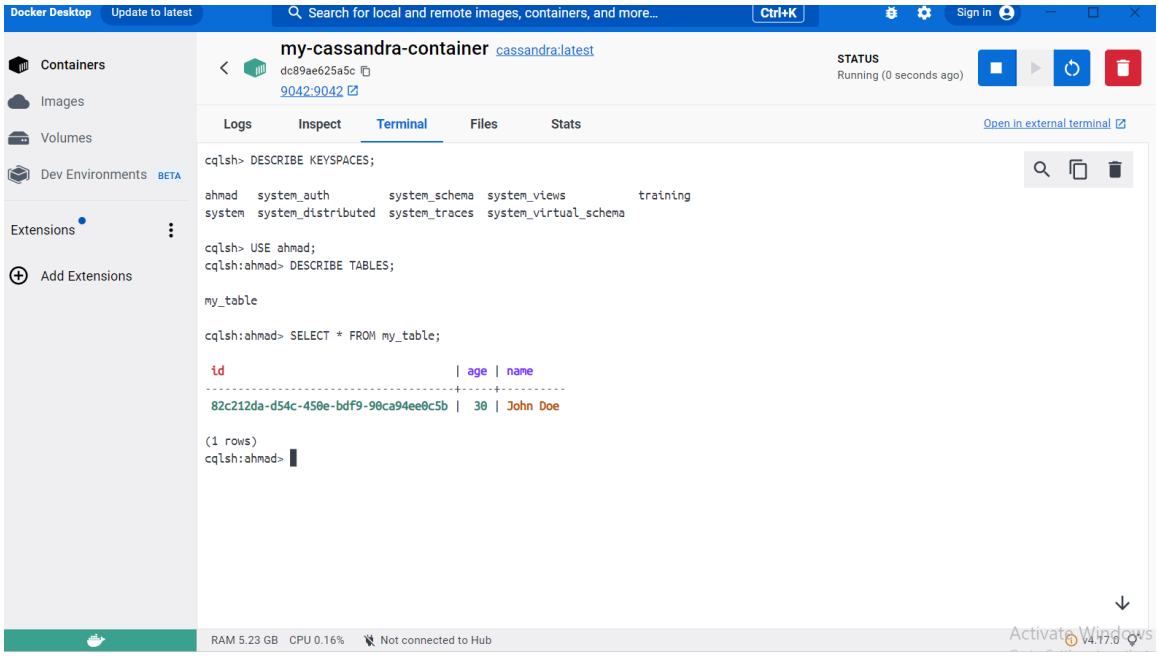
In this figure, we can see that we ran the Cassandra database, and ran **"cqlsh" which is important to apply important commands such as creating key spaces, tables, inserting data into tables and etc.**

```
cqlsh> CREATE KEYSPACE Training WITH replication = {'class': 'SimpleStrategy', 'replication_factor':1};
cqlsh> DESCRIBE KEYSPACES;

ahmad   system_auth         system_schema  system_views           training
system  system_distributed  system_traces  system_virtual_schema

cqlsh>
```

In this figure, I created a new key space which is called “Training” as shown above, and also I displayed all the key spaces that I used.



Here I applied more commands so, I described the key spaces and used “ahmad” key space and described the tables that are created in it, so as shown we can see that there is a table called my_table which I created and inserted some data into it, and finally, I used the command “SELECT * FROM my_table” to show all the data that are filled in this table.

EIGHTH WEEK (LAST WEEK)

This week I made solved many, and many tasks from which I would be sure about dealing with all the new things that I learned, so I solved many tasks using Scala, Spark and Cassandra.

And now I will include some of my tasks and also I will explain them briefly.

-FIRST TASK

The purpose of this task is to find the top 3 most popular products in the given Countries on the given input file (Top 3 according to the Total Revenue).

```
val Reading_Configuration = spark.
  read.
  option("inferSchema","true").
  option("header", "true").
  csv( path = "C:\\TestFiles\\homework.csv")
val revenue = Reading_Configuration.withColumn( colName = "revenue",
  col( colName = "Quantity") * col( colName = "UnitPrice"))
```

Here as we can see, I've started by initializing a variable that is responsible for reading the given file which is called here as we can see (homework.csv).

After reading the file, I wanted to find the revenue for each input data in the file and to make sure that I see the result of the new data, I used the "WithColumn" function, this function creates a new column, and stores the result of the applied operation, and in this case, the operation is to find the revenue for each product by multiplying the value of the quantity with the value of the UnitPrice.

homework.csv - Notepad

File Edit Format View Help

```
InvoiceNo,StockCode,Description,Quantity,UnitPrice,Country
536365,85123A,White hanging heart t-light holder,6,2.55,United Kingdom
536365,71053,White metal lantern,6,3.39,United Kingdom
536365,84406B,Cream cupid hearts coat hanger,8,2.75,United Kingdom
536366,22633,Handmade mini  crochet  rose,6,1.85,France
536366,22632,Handmade chrochet  butterfly,2,2.1,France
536367,84879,Assorted colour bird ornament,32,1.69,United Kingdom
536367,22745,POPPY'S PLAYHOUSE BEDROOM ,6,2.1,United Kingdom
```

As we can see here, this is the input file that I've read in the above code, and it shows some features such as Quantity, UnitPrice, Country and etc...

After that we need to groupBy data by the column “Country” and “StockCode”, then we need to find the sum of the revenue and here we will get the total revenue as shown below.

```
val GroupbyCountryAndStockcode = revenue.groupBy(col( colName = "Country"), col( colName = "StockCode")).
  agg(round(sum( columnName = "revenue"), scale = 1).
    alias( alias = "Total_Revenue"))

val windospec = Window.partitionBy(col( colName = "Country")).
  orderBy(col( colName = "Total_Revenue").desc)

val Resulting = GroupbyCountryAndStockcode.withColumn( colName = "rank", rank.over(windospec)).
  filter(col( colName = "rank") <= 3)
  .select(col( colName = "Country"), col( colName = "StockCode"),col( colName = "Total_Revenue"))
```

Then after that, we need to use the filter function and that to filter the top 3 revenues for the best 3 products in each country, and that is used by the “filter” function.

```
Resulting.show()
```

And here, this line of code is responsible for loading the data and printing it on the console.

```
Run:  Revenue
|       Country|StockCode|Total_Revenue|
+--------------+---------+-------------+
|        France|    22633|         11.1|
|        France|    22632|          4.2|
|United Kingdom|    84879|         54.1|
|United Kingdom|   84406B|         22.0|
|United Kingdom|    71053|         20.3|
+--------------+---------+-------------+
Version Control  Run  TODO  Problems  Terminal  Endpoints  Services  Profiler  Build  sbt shell
Build completed successfully in 2 sec, 974 ms (moments ago)   TestFiles
```

And here we can see the result of the above code showing the total revenue for the top 3 popular products that have the most “total revenues”.

-SECOND TASK

In this task, the purpose was to print many results such as **finding the total revenue for each customer, total revenue for each product, average transaction per customer, daily revenue, and the day with the highest revenue.**

First of all, we need to read the file to deal with data as in the previous example.

```
val Reading_Configuration = spark.read.option("inferSchema","true").
  option("header" , "true")
  .csv( path = "C:\\TestFiles\\Task.csv")
```

So this line will help us to read the file, and make it easy to deal with the data inside it.

Task.csv - Notepad

File Edit Format View Help

```
transaction_id,customer_id,product_id,quantity,price,timestamp
1,1001,1,2,10.0,2022-03-20 10:30:00
2,1002,2,3,12.5,2022-03-20 11:15:00
3,1003,3,1,20.0,2022-03-20 12:00:00
4,1001,1,1,10.0,2022-03-20 12:30:00
5,1002,1,2,10.0,2022-03-21 09:00:00
6,1003,2,2,12.5,2022-03-21 10:00:00
7,1001,3,3,20.0,2022-03-22 11:00:00
8,1002,1,4,10.0,2022-03-22 11:30:00
9,1003,1,1,10.0,2022-03-22 12:00:00
10,1001,2,2,12.5,2022-03-23 10:00:00
11,1002,2,1,12.5,2022-03-23 11:00:00
12,1003,3,3,20.0,2022-03-23 12:00:00
```

This file includes some data that will be important for us to deal with such as **"transaction_id"**, **"customer_id"**, **"product_id"**, **"quantity"**, **"price", and "timestamp".**

```
val Revenue = Reading_Configuration.
  withColumn( colName = "revenue", col( colName = "quantity") * col( colName = "price"))

val Customer_Revenue = Revenue.
  groupBy( col1 = "customer_id").
  agg(sum( columnName = "revenue").
    alias( alias = "Total_Revenue_For_Each_Customer"))

Customer_Revenue.show()
```

Here, I'm applying some operations on the file to find the Total revenue for each customer.

As we can see, **I used the groupBy function, agg function, and the sum function.**

And for the result, I used the "show ()" function to load the data and print it on the console as shown below.

```
+-----------+-------------------------------+
|customer_id|Total_Revenue_For_Each_Customer|
+-----------+-------------------------------+
|       1002|                          110.0|
|       1001|                          115.0|
|       1003|                          115.0|
+-----------+-------------------------------+
```

```
val Product_Revenue = Revenue.groupBy( col1 = "product_id")
  .agg(sum( columnName = "revenue")
    .alias( alias = "Total_Revenue_For_Each_Product"))

  Product_Revenue.show()
```

Here, I used the same thing, but instead, I applied it to the column "product_id" to find the total revenue for each product, and the output is shown below.

```
|product_id|Total_Revenue_For_Each_Product|
+----------+------------------------------+
|         1|                         100.0|
|         3|                         140.0|
|         2|                         100.0|
+----------+------------------------------+
```

```
val Average = Revenue
  .groupBy(col( colName = "customer_id"), col( colName = "transaction_id"))
  .agg(sum( columnName = "revenue").alias( alias = "transaction_sum"))
  .groupBy( col1 = "customer_id")
  .agg(avg( columnName = "transaction_sum")
    .alias( alias = "average_transaction_per_customer"))
Average.show()
```

In this section of code, I wanted to find the "Average transaction for each customer", and in this case, I had to apply many operations.

And as we can see I used the groupBy function to group the data based on "customer_id" and "transaction_id", then I used the agg function to apply the sum operation, and I used it to find the sum of the revenue.

Then I used the groupBy function again to groupBy the data based on the "customer_id" column and finally to find the average for the transaction sum that I calculated.

Note: the *alias ()* function is used to rename the column so that it would be easy for the programmer to understand the output.

And here is the output for the previous code.

```
|customer_id|average_transaction_per_customer|
+-----------+--------------------------------+
|       1002|                            27.5|
|       1001|                           28.75|
|       1003|                           28.75|
+-----------+--------------------------------+
|
```

```
val Day_Revenue = Revenue.groupBy(date_trunc( format = "day",  timestamp = $"timestamp").alias( alias = "day"))
  .agg(sum( columnName = "revenue")
    .alias( alias = "daily_revenue"))

Day_Revenue.show()
```

In this section, I wanted to find the daily revenue (the revenue for each day), so I used the same functions to express the operations and to deal with them in such an easy and flexible way.

And here is the output of the previous section.

```
+-------------------+-------------+
|                day|daily_revenue|
+-------------------+-------------+
|2022-03-21 00:00:00|         45.0|
|2022-03-22 00:00:00|        110.0|
|2022-03-23 00:00:00|         97.5|
|2022-03-20 00:00:00|         87.5|
+-------------------+-------------+
```

```
val Resulting = Day_Revenue.orderBy(desc( columnName = "daily_revenue")).select( col = "day").first()
println(s"Day with the highest revenue: $Resulting")
```

And here finally, I was able to find the Day with the highest revenue, and that was easy because, in this section, I just had to deal with the previous section which was “Calculating the Daily Revenue”, and I had just to order the data in a descending way, and then choose the first element (which obviously it’s the day with the highest revenue).

And the output of the previous sections is shown below.

```
Day with the highest revenue: [2022-03-22 00:00:00.0]
```

-THIRD TASK

In this task, I used Cassandra with Spark and Scala as shown below.

```
object Cassandra {
  def main(args: Array[String]) {

    val conf = new SparkConf()
      .setAppName("PrintTable")
      .setMaster("local[2]")
      .set("spark.cassandra.connection.host", "localhost")

    val sc = new SparkContext(conf)

    val rdd = sc.cassandraTable( keyspace = "ahmad",  table = "my_table")


    val header = "id,name,age"
    val data = rdd.collect().map(_.toString.drop(1).dropRight(1))
    val output = (header +: data).mkString("\n")

    println(output)
```

In this code, I configured the main configurations to start the connection with Cassandra and also I started by configuring the main headers in the table which they “id, name, age”.

Also, I initialized an **Rdd to connect the server to the key space and the table that is associated with that key space which is “Ahmad”.**

Finally, I loaded out the output of the following code as shown below.

```
id,name,age
assandraRow{id: 82c212da-d54c-450e-bdf9-90ca94ee0c5b, age: 30, name: John Doe
```

This figure shows the output of the previous code.

Finally, to make sure that this is the data stored in the database and in the table, I opened **Docker** and loaded the data for the "Ahmad" table as shown below.

```
# cqlsh
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.0 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> DESCRIBE KEYSPACES;

ahmad   system_auth         system_schema  system_views           training
system  system_distributed  system_traces  system_virtual_schema

cqlsh> USE ahmad;
cqlsh:ahmad> DESCRIBE TABLES;

my_table

cqlsh:ahmad> SELECT * FROM my_table;

 id                                   | age | name
--------------------------------------+-----+----------
 82c212da-d54c-450e-bdf9-90ca94ee0c5b |  30 | John Doe

(1 rows)
```

Additional Links that Rand asked me to cover.

-https://www.infoworld.com/article/3236869/what-is-apache-spark-the-big-data-platform-that-crushed-hadoop.html

-https://www.analyticsvidhya.com/blog/2016/11/an-introduction-to-clustering-and-different-methods-of-clustering/

-https://www.informit.com/articles/article.aspx?p=2928186

-instaclustr.com/support/documentation/cassandra-add-ons/apache-spark/getting-started-with-spark-jobserver-and-instaclustr/

-https://stackoverflow.com/questions/32621990/what-are-workers-executors-cores-in-spark-standalone-cluster